

Responsibility Specialization in LLM-Based Multi-Agent Systems

A Controlled, Mechanism-Level Study Isolating the Causal Effect of Role-Based Responsibility Specialization on Reliability, Cost, Latency, and Failure Pathways

Technical Research Report

Internship Research Project 2026

Part 1 — Empirical GitHub Characterization | Part 2 — Controlled Experiment

Date: 30 September 2026

Abstract

Large Language Model (LLM)-based multi-agent systems increasingly rely on architectural mechanisms — most prominently role-based responsibility specialization — to decompose complex tasks across cooperating agents. While a growing body of work compares whole architectures and correlates architectural properties with performance, there is little controlled, mechanism-level experimentation that isolates the effect of responsibility specialization alone while holding every other major architectural factor constant. This report addresses that gap in two parts. Part 1 characterizes how specialization and coordination actually appear in the wild by mining 504 open-source repositories and extracting 333 candidate multi-agent systems through evidence-grounded static analysis. Part 2 presents a pre-registered, within-task controlled experiment that manipulates responsibility specialization as the sole independent variable in an otherwise byte-identical three-agent linear pipeline (generalist versus specialized), across 30 tasks $\times$ 5 runs $\times$ 2 conditions (300 runs) on a single fixed model (qwen2.5:7b). Using paired task-level Wilcoxon signed-rank tests with Holm correction, we find that specialization preserves task accuracy (86.7% vs 86.0%, not significant) while significantly reducing token cost by $\sim 29\%$ ($p < 1e-5$) and latency by $\sim 23\%$ ($p = 0.019$). At the same time, specialization introduces a new, frequent failure mode — role-boundary violations (wrong-responsibility execution and out-of-role tool use) present in 52–77% of specialized runs — that is structurally impossible in the generalist condition yet, in this setting, benign to final correctness. The results provide clean causal evidence that responsibility specialization is primarily an efficiency mechanism rather than an accuracy mechanism in short linear pipelines, and that instruction-based role boundaries are only imperfectly obeyed.

Keywords: LLM agents; multi-agent systems; responsibility specialization; controlled experiment; reliability; token cost; latency; failure taxonomy; role adherence.

1. Problem Statement

LLM-based multi-agent systems coordinate several LLM-driven agents to solve tasks that are hard for a single monolithic prompt. A central design lever in these systems is responsibility specialization: rather than giving every agent the full set of capabilities, the designer assigns narrower, role-specific responsibilities — for example, one agent retrieves information, a second solves, and a third verifies. Specialization is widely assumed to improve modularity, reduce prompt bloat, and sharpen each agent's behavior. Yet these assumed benefits are rarely tested in isolation.

This work began from four broad questions about LLM-based multi-agent systems: (1) what architectures are used; (2) how responsibilities are distributed among agents; (3) how architectural choices affect reliability, cost, and latency; and (4) why such systems succeed or fail. A review of the literature made clear that all four are shaped by the same underlying theme — that multi-agent behaviour depends not only on the individual agents but on how responsibilities and interactions are organized — and it allowed us to narrow this broad space to a single, controllable mechanism: responsibility specialization.

The core difficulty is confounding. In practice, when practitioners adopt a specialized architecture they simultaneously change many factors at once: the number of agents, the communication topology, the tools each agent may call, the prompt content and its quality, and sometimes even the underlying model. When a specialized system then performs differently from a generalist baseline, it is impossible to attribute the difference to specialization per se, because specialization was never varied on its own. Existing empirical work — including architecture surveys, benchmark comparisons, and correlational studies that link architectural attributes to outcomes — tells us which whole systems perform well, but not why, at the level of the individual mechanism.

This report therefore targets a precise, answerable question:

What is the causal effect of responsibility specialization — allocating distinct task responsibilities to separate agents via role-specific instructions and bounded scopes — on an LLM-agent system's task reliability, resource consumption, latency, and behavioral failure pathways, when all other major architectural factors are held constant?

Answering this requires more than another architecture comparison. It requires a controlled manipulation in which specialization is the only thing that changes between conditions, so that any measured difference in reliability, cost, latency, or failure behavior can be causally attributed to the mechanism itself. That is the contribution of this work.

2. Literature Review

2.1 LLM multi-agent systems and the role of specialization

Multi-agent LLM frameworks such as AutoGen, MetaGPT, CAMEL, CrewAI, and orchestration libraries like LangGraph popularized the idea of decomposing a task across role-playing agents (e.g., "researcher", "coder", "reviewer"). The animating hypothesis is a division-of-labor argument borrowed from human organizations: narrowing each agent's remit should reduce cognitive load per step, make behavior more predictable, and allow specialized prompting or tooling. A parallel line of work on prompting and self-verification (chain-of-thought reasoning, self-consistency, and reflection/critic loops) suggests that separating generation from verification can catch errors, which is often cited as motivation for dedicated verifier agents.

2.2 What existing evaluations do — and do not — establish

Empirical studies of multi-agent systems predominantly evaluate architectures as wholes. They compare, for instance, a hierarchical planner-executor design against a flat debate design on a benchmark, or they run ablations that add or remove an entire agent. Such studies establish useful end-to-end facts and have identified architectural factors — topology, number of agents, presence of a critic, memory — that correlate with accuracy and reliability. However, because these factors co-vary in real systems, the evidence is largely correlational at the mechanism level. Recent reliability-focused work also documents characteristic multi-agent failure modes (inter-agent information loss, redundant work, cascading errors, and specification/role misalignment), but again typically observes them across differing systems rather than under a single controlled manipulation.

Four strands of recent work were especially influential in shaping this study; they are summarized in Table 1, together with what each contributed to our thinking.

Key work	What it studied	Key outcome	What we took from it
Science of Scaling Multi-Agent Systems	Effect of scaling the number of agents and system organization	More agents do not automatically improve performance	Organization, not agent count, dictates outcomes
Why Do Multi-Agent LLM Systems Fail?	Failure mechanisms in multi-agent systems	Failures emerge from agent interactions and coordination	Interactions create distinct failure pathways
MAESTRO	System-level multi-agent evaluation	Architecture and coordination affect system behaviour	System topology matters and must be controlled
AgentRx & AgentArc	Agent architecture, behaviour, and design	Agent organization is an important system-	The mechanisms themselves need to be

		level factor	isolated
--	--	--------------	----------

Table 1. Representative literature and its influence on this study. Overall takeaway: multi-agent behaviour depends not only on individual agents, but on how responsibilities and interactions are organized.

2.3 Evidence from open-source practice (Part 1 of this project)

To ground the study in how specialization is actually deployed, Part 1 of this project mined 504 open-source GitHub repositories through a nine-stage, evidence-preserving pipeline (GitHub search → clone and source-code mining → agent and workflow detection → evidence extraction → candidate identification → semantic classification → gold validation → final dataset), guided by the principle "extract evidence from code; don't decide the research variable prematurely." This yielded 333 candidate multi-agent systems comprising 1,087 agents, of which 964 were responsibility-bearing, connected by 296 explicit interaction edges. Each system's coordination architecture and degree of specialization were then labeled using a hybrid method: architecture assigned by a deterministic precedence rule over extracted structural evidence, and specialization labeled by an LLM classifier. Validation against 10 gold cases (spanning handoff, hierarchical, parallel, sequential, feedback, conditional-graph, event-driven, nested-team, and framework-control patterns) scored 9/9 correct on architecture, exact on agent boundaries, and ~75% agreement on specialization (errors within ± 1 level and non-directional).

Under evidence-grounded classification, responsibility specialization was distributed as High 62%, Medium 26%, Low 8%, and Unresolved 4%; coordination architecture as Unresolved 34.5%, Sequential 31.2%, Conditional Routing 18.0%, Hierarchical 9.6%, Parallel 3.9%, Iterative Feedback 2.1%, and Event-driven 0.6%.

Two findings from that corpus directly motivate the controlled experiment. First, specialization is pervasive: the majority of systems (62% High) were labeled as highly specialized, confirming that responsibility specialization is a mainstream, high-impact design choice worth isolating. Second, coordination structure is frequently under-evidenced: 34.5% of systems had no extractable structural evidence at all ("unresolved"), and an early LLM-only classifier had massively over-predicted "sequential" coordination (inflating it to ~66% before evidence-grounding corrected it to 31.2%). The methodological lesson — that LLM judgments of architecture are biased and must be evidence-grounded, retaining honest "unresolved" labels rather than forcing inaccurate ones — carries directly into Part 2's insistence on deterministic checkers and measured, rather than assumed, behavior.

A guardrail from Part 1 is worth stating explicitly for interpretation: prevalence figures describe the sampled open-source corpus (which is skewed by a few large frameworks), not real-world multi-agent systems in general. The corpus establishes that specialization

matters and is common; it does not by itself quantify specialization's causal effect. That quantification is the job of Part 2.

2.4 The gap

Synthesizing the above: the literature strongly motivates responsibility specialization and characterizes system-level outcomes and failure modes, and the open-source corpus confirms that specialization is widespread. What is missing is controlled, mechanism-level experimentation that isolates the effect of responsibility specialization while holding model, agent count, topology, tools, inference parameters, tasks, and evaluation constant. Without such isolation, the field cannot say whether specialization's benefits come from the division of responsibility itself or from the many other choices bundled with it. This report closes that gap.

3. Proposed Solution

We propose a controlled, pre-registered, within-task experiment that treats responsibility specialization as a single manipulable variable and measures its causal effect on a fixed task suite. The experiment is designed to answer the refined research question along three axes: (1) Effect — how does specialization change outcomes?; (2) Mechanism — why does specialization change behaviour?; and (3) Boundary conditions — when does it help or hurt? Part 1 tells us what is observed in practice; Part 2 tests why it matters. The design rests on three principles.

3.1 One mechanism, isolated

Both experimental conditions run the exact same three-agent, strictly linear pipeline (agent A $\rightarrow$ agent B $\rightarrow$ agent C, two handoffs, no loopback). The system is built from a shared set of instruction blocks corresponding to three responsibilities — Retrieve (R), Solve (S), and Verify (V). The only difference between conditions is how those identical blocks are allocated across agents:

- Generalist: every one of the three agents holds all three responsibility blocks {R, S, V}.
- Specialized: agent 1 holds {R}, agent 2 holds {S}, agent 3 holds {V}.

Crucially, the manipulation is implemented purely as allocation of byte-identical instruction blocks. The union of instruction content is identical across conditions; only who holds which block differs. This isolates specialization from any difference in prompt content, wording, or quality — a confound that plagues naturalistic comparisons — because the same sentences are simply distributed differently. All remaining code (the agent loop, pipeline, LLM interface, tools, evaluator, and failure classifier) is shared by both conditions, so the two arms cannot silently diverge.

3.2 Measure, don't block, role violations

Both conditions give every agent access to the same tools (a deterministic lookup tool and a calculator). We deliberately do not enforce specialization by restricting tool access. If a specialized "retriever" chooses to solve the whole task or call the calculator, it is allowed to — and that overstep is recorded as a measured failure (wrong-responsibility execution or out-of-role tool use). This makes role adherence an empirical outcome rather than a hard constraint, which is exactly what is needed to study specialization's real behavioral consequences.

3.3 Deterministic, honest evaluation

Correctness is judged by deterministic checkers rather than an LLM judge, removing a well-known source of bias (and echoing Part 1's finding that LLM meta-judgments are unreliable). Because the five runs per task are not independent, the primary statistical analysis is paired at the task level. The full design — conditions, tasks, tools, parameters, hypotheses, and analysis plan — was pre-registered and frozen before the confirmatory runs, so the results are confirmatory rather than post-hoc.

4. Methodology

4.1 Experimental design

The study is a two-condition (generalist vs specialized), within-task comparison. Every task is run under both conditions, enabling paired analysis. The scale is 30 tasks $\times$ 5 runs per task $\times$ 2 conditions = 300 runs.

4.2 Independent variable and factors held constant

The single independent variable is responsibility-block allocation (generalist vs specialized). Everything else is held constant and shared, as summarized below.

Factor held constant	Setting
Model	qwen2.5:7b (single fixed instruct model, local via Ollama)
Agent count	3
Topology / control flow	Strict linear A→B→C, exactly 2 handoffs, no loopback
Tools (both conditions)	lookup (deterministic), calculator — available to all agents
Temperature / top_p	0.7 / 0.9
Context / output budget	num_ctx 8192; num_predict 1024; max 4 tool iters per agent

Tasks	30 frozen tasks (10 easy, 12 medium, 8 hard)
Runs per task / seeds	5 runs; seeds [11, 22, 33, 44, 55] by run index
Evaluation & failure classifier	Deterministic; identical and blind to condition

Table 2. Factors held constant across both conditions. Only responsibility-block allocation varies.

The model choice deserves a note. An initial pilot on llama3:8b exhibited floor effects (generalist 17% / specialized 0% accuracy), which would have made the conditions indistinguishable for the wrong reason. Switching to qwen2.5:7b lifted both conditions off the floor (~83% in the pilot) and was then frozen. The model is a held-constant factor, not a variable.

4.3 Task suite and retrieval design

The 30 tasks require retrieving a fact from a provided context passage, performing a computation or reasoning step, and reporting a final answer — a shape that genuinely exercises all three R/S/V responsibilities. Retrieval is deterministic: the lookup tool returns the task's own context passage, so retrieval quality is not a hidden source of noise and both conditions receive identical information and opportunity. Tasks span three difficulty levels to support a difficulty $\times$ condition analysis.

4.4 Dependent variables

- Reliability (primary): final-answer correctness by deterministic check.
- Cost: input / output / total tokens (exact from the model runtime) and number of LLM calls; system-prompt overhead reported separately from generated tokens.
- Latency: wall-clock seconds per run.
- Mechanism-level: handoffs, repeated work (inter-agent output overlap), corrections, and a nine-category failure taxonomy (F1–F9).

The failure taxonomy, applied identically and blind to condition, is defined as follows.

Code	Failure mode
F1	Wrong-responsibility execution (agent acts outside its assigned scope)
F2	Retrieval failure (a required fact never surfaced by the first agent)
F3	Reasoning/solving failure (solve-stage proposed answer is wrong)
F4	Verification failure (verify stage let an error through or broke a correct answer)
F5	Handoff/information-loss (fact present early, lost by the final agent)
F6	Redundant/repeated work (high output overlap between agents)
F7	Unnecessary/out-of-scope tool use
F8	Inter-agent correction (final answer changed vs. the solve stage)

F9	Final-answer failure (system output incorrect)
----	--

Table 3. F1–F9 failure taxonomy. Note that F1 and F7 are structurally only possible under specialization, since a generalist agent owns all responsibilities and cannot act out of role.

4.5 Statistical analysis plan

Because the five runs per task are not independent, the primary analysis is a paired task-level Wilcoxon signed-rank test across the 30 tasks, computed separately for each of the three primary dependent variables (success, total tokens, latency), with Holm correction applied across those three tests. Reliability is additionally reported with task-mean bootstrap 95% confidence intervals and broken down by difficulty. A secondary GEE logistic model (correctness $\sim$ condition, clustered by task) provides a model-based robustness check on the reliability result; it complements, and does not override, the primary paired analysis.

4.6 Reproducibility and integrity safeguards

- Pre-registration: conditions, tasks, taxonomy, and analysis plan were frozen before confirmatory runs.
- No performance tuning: prompts were never tuned to improve accuracy from results; only pre-freeze bug fixes and measurement-instrument calibration were permitted.
- Crash-safe execution: each completed run is appended and fsync'd to JSONL and mirrored to SQLite; re-running skips completed runs and resumes, so an interruption loses at most one in-flight run.
- Pilot data archived separately and never pooled with confirmatory data.

5. Results

All 300 confirmatory runs completed (150 per condition). Three runs initially timed out under transient CPU contention (Ollama call exceeding the 300s timeout); the crash-safe runner logged and skipped them, and a resume pass filled all three (completing in 51–165s), yielding a full $n = 300$.

5.1 Reliability (H1): no effect

Responsibility specialization did not change task accuracy. The generalist condition achieved 86.7% and the specialized condition 86.0%; the median paired difference (specialized – generalist) was exactly zero. The Wilcoxon signed-rank test was not significant (raw $p = 0.497$; Holm-adjusted $p = 0.497$). The task-mean bootstrap 95% confidence intervals overlap almost entirely (generalist 0.767–0.947; specialized 0.760–0.947). Accuracy was also indistinguishable at every difficulty level (easy 0.90/0.90, medium 0.85/0.833, hard 0.85/0.85), giving no evidence that specialization helps or hurts on harder tasks. The pre-registered secondary analysis — a GEE logistic model of correctness on condition, clustered by task over all 300 runs — corroborates this null

result: the specialization coefficient is -0.057 (robust SE 0.202, $z = -0.28$, $p = 0.780$), i.e. no significant change in the odds of a correct answer.

5.2 Cost (H2): specialization is substantially cheaper

Specialization significantly reduced token cost. Mean total tokens fell from 4,147 (generalist) to 2,958 (specialized) — roughly a 29% reduction — with a median paired difference of $-1,010$ tokens (Wilcoxon raw $p = 3.18\text{e-}6$; Holm-adjusted $p = 9.5\text{e-}6$). A large part of this is structural and expected: system-prompt overhead alone fell from 1,602 to 1,088 tokens, because the generalist carries all three R/S/V instruction blocks in all three agents by design. This is a genuine, transparent consequence of specialization rather than a confound.

5.3 Latency (H3): specialization is faster

Specialization also reduced wall-clock latency, from a mean of 89.9s (generalist) to 69.4s (specialized) — about a 23% reduction — with a median paired difference of -12.8 s (Wilcoxon raw $p = 0.0093$; Holm-adjusted $p = 0.0186$). The specialized condition also made slightly fewer LLM calls on average (5.60 vs 5.96).

Dependent variable	Generalist	Specialized	Median diff (spec-gen)	Holm-adj. p	Verdict
Success rate	86.7%	86.0%	+0.000	0.497	No effect
Total tokens (mean)	4,147	2,958	$-1,010$	$9.5\text{e-}6$ *	Specialized cheaper ($\sim 29\%$)
System-prompt tokens	1,602	1,088	—	—	Lower by design
Latency (mean, s)	89.9	69.4	-12.8	0.0186 *	Specialized faster ($\sim 23\%$)
LLM calls (mean)	5.96	5.60	—	—	Slightly fewer

Table 4. Primary results. Paired task-level Wilcoxon signed-rank across $n = 30$ tasks; * denotes significance after Holm correction across the three primary DVs.

5.4 Failure pathways (H4–H6): a new failure mode, benign here

The failure-taxonomy analysis reveals the most interesting mechanistic result. Two failure types — F1 (wrong-responsibility execution) and F7 (out-of-role tool use) — appear only in the specialized condition and are, by construction, impossible in the generalist condition (where every agent owns every responsibility). In the specialized arm they are frequent: F1 occurred in 76.7% of runs and F7 in 52.7% of runs. In other words, specialized agents routinely overstep their assigned role boundaries — the retriever solves, agents reach for tools outside their remit — yet, as Section 5.1 showed, the

pipeline still matches generalist accuracy. Role-boundary violations are therefore common but, in this short linear setting, benign to the final outcome.

Other failure categories shifted modestly rather than dramatically. Redundant/repeated work (F6) was lower under specialization (52.7% vs 62.0% of runs), consistent with narrower roles reducing overlap. Retrieval-related failures (F2) were common in both conditions (75.3% specialized, 78.7% generalist), reflecting a strict required-information heuristic rather than a between-condition difference. Inter-agent corrections (F8) were somewhat more frequent under specialization (21.3% vs 11.3%), as one might expect from a dedicated verify stage acting on a separate solve stage.

Failure	Generalist (% runs)	Specialized (% runs)	Interpretation
F1 wrong-responsibility	0% (impossible)	76.7%	New failure mode from specialization
F7 out-of-role tool use	0% (impossible)	52.7%	New failure mode from specialization
F6 redundant work	62.0%	52.7%	Reduced by narrower roles
F8 inter-agent correction	11.3%	21.3%	Dedicated verify stage more active
F2 retrieval failure	78.7%	75.3%	Similar; strict info heuristic
F5 information loss	10.7%	18.7%	Slightly higher across handoffs
F9 final-answer failure	13.3%	14.0%	$\approx$ equal (mirrors accuracy)

Table 5. Selected failure-pathway frequencies. F1 and F7 are structurally exclusive to the specialized condition and are reported as a newly introduced failure mode, not as evidence that specialization is 'worse'.

5.5 Figures

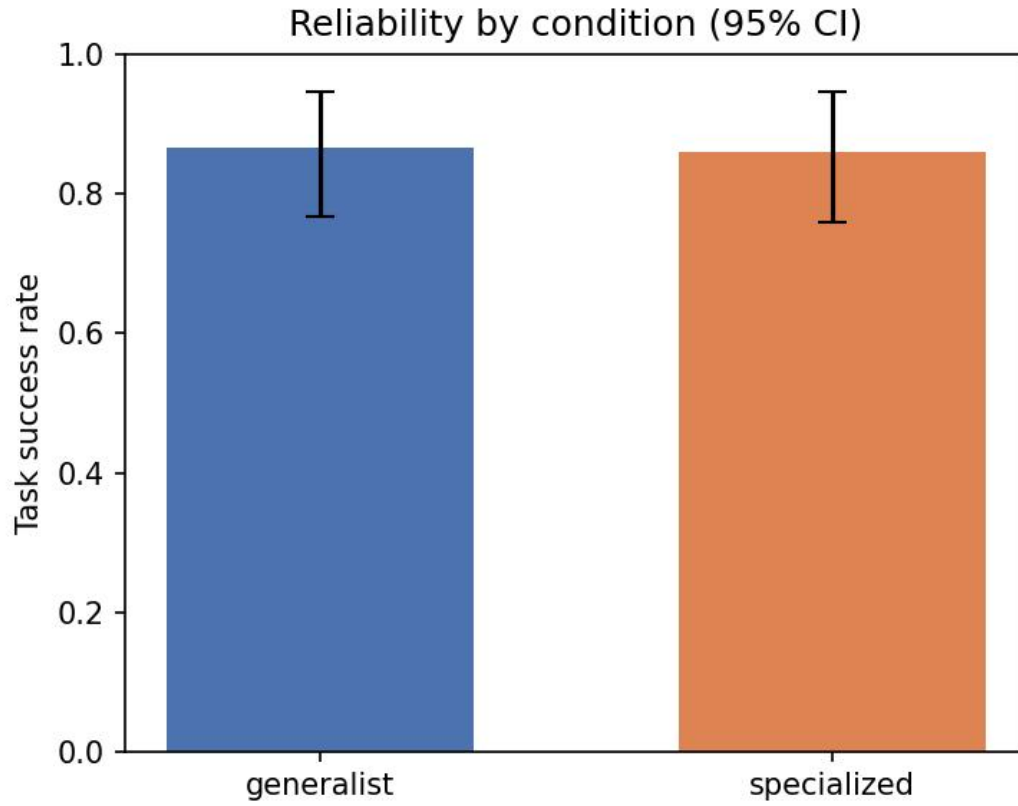


Figure 1. Task success rate by condition with task-mean bootstrap 95% confidence intervals. The two conditions are statistically indistinguishable (86.7% vs 86.0%).

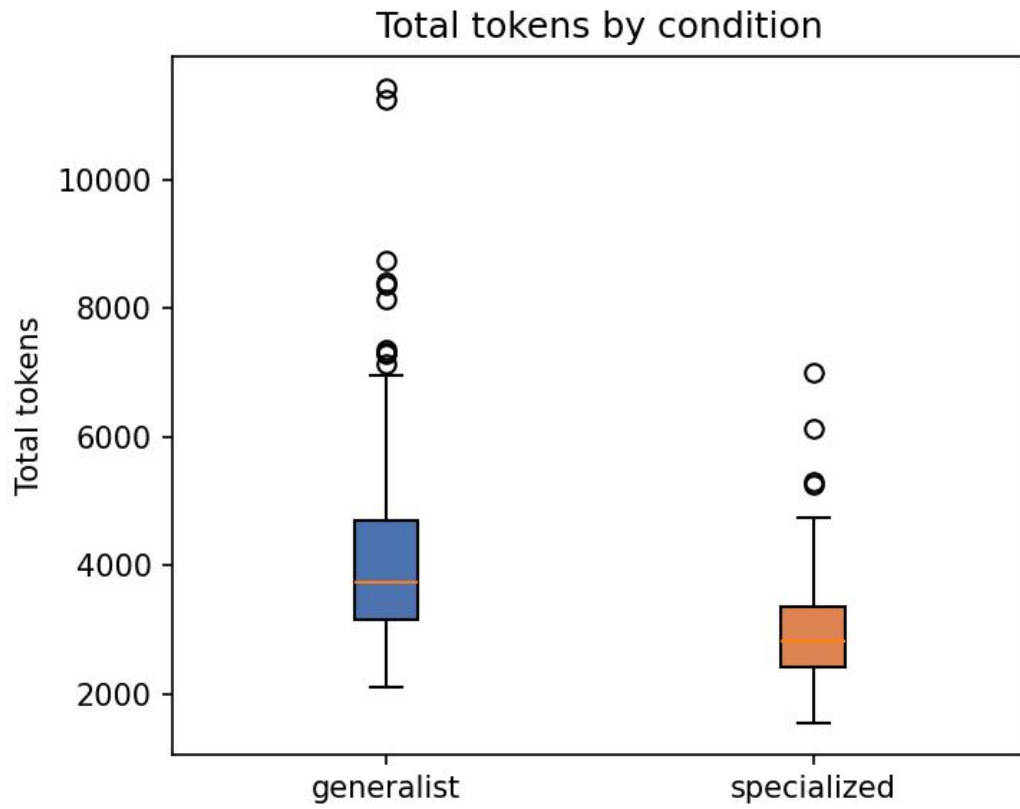


Figure 2. Total token cost by condition. Specialization reduces mean total tokens by ~29% (Holm-adjusted $p = 9.5e-6$).

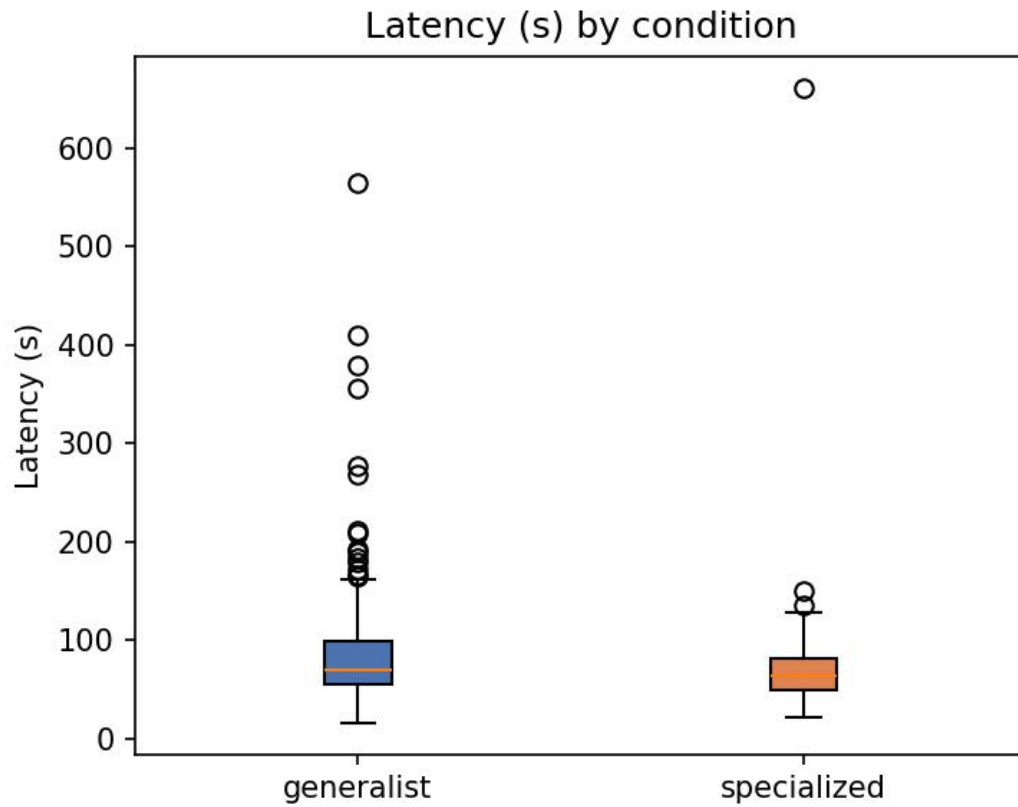


Figure 3. Wall-clock latency by condition. Specialization reduces mean latency by ~23% (Holm-adjusted $p = 0.0186$).

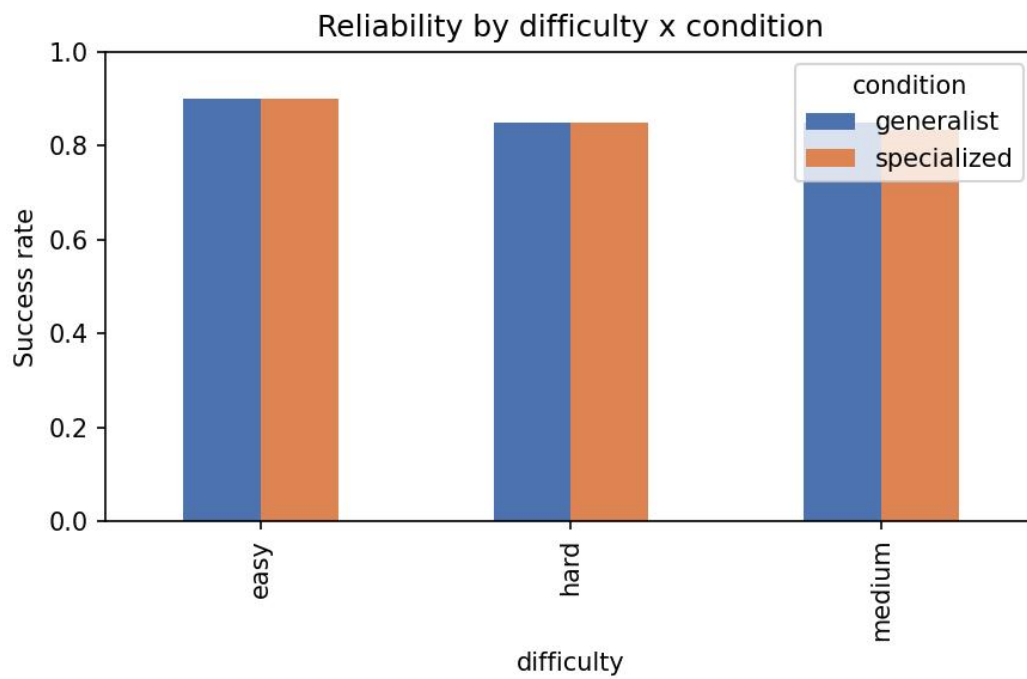


Figure 4. Reliability by difficulty $\times$ condition. No condition advantage emerges at any difficulty level.

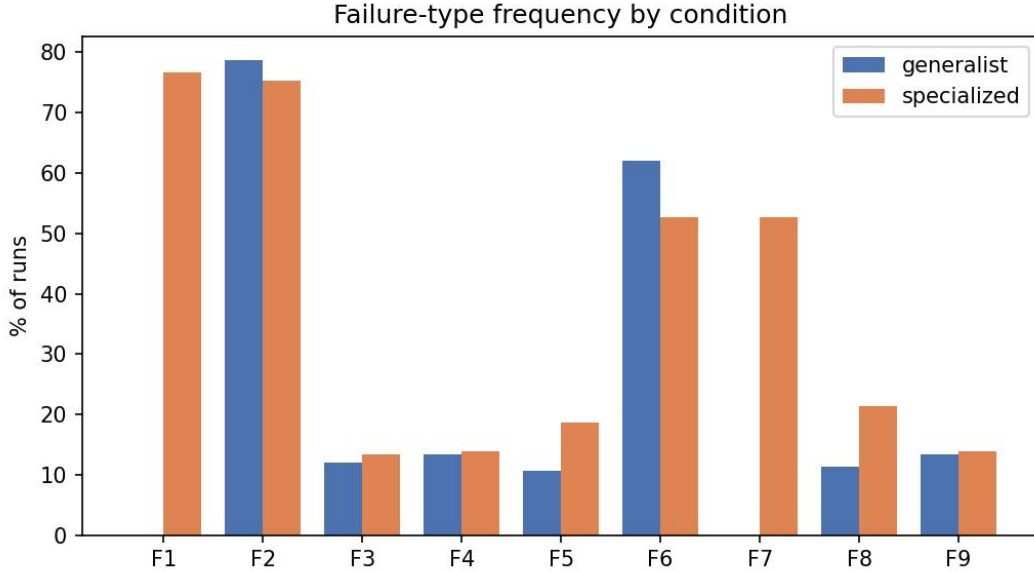


Figure 5. Failure-type frequency per condition. F1 and F7 appear only under specialization; F6 (redundant work) is lower under specialization.

6. Discussion

The experiment yields a clean, causal picture within its scope. Because responsibility specialization was the only factor varied — implemented as a re-allocation of byte-identical instruction blocks over a shared codebase — the observed differences can be attributed to the mechanism itself rather than to confounded changes in prompts, tools, model, or topology.

The headline interpretation is that, in a short three-agent linear pipeline, responsibility specialization is primarily an efficiency mechanism, not an accuracy mechanism. It leaves task reliability unchanged while cutting token cost by roughly a third and latency by roughly a quarter — chiefly because generalist agents each carry the full instruction set, inflating prompt overhead and generation. For cost- and latency-sensitive deployments where accuracy is already acceptable, specialization is attractive on economic grounds alone.

The second contribution is behavioral. Instruction-based role boundaries are only imperfectly obeyed: specialized agents violated their assigned scope in the majority of runs. This is a genuine finding about how LLM agents follow role instructions, and it complements Part 1’s evidence that LLM self-descriptions of architecture are unreliable. Importantly, in this setting the violations did not harm outcomes — but that benign result is specific to a short pipeline with deterministic retrieval and simple tasks; in longer pipelines, or where out-of-role actions can corrupt shared state, the same violation rate

could become costly. If strict role adherence is required, these results imply it must be enforced structurally (e.g., by restricting tool access), not merely requested in the prompt.

6.1 Limitations

- Single model and single topology: results are established for qwen2.5:7b on a strict three-agent linear pipeline; generalization to other models, larger agent counts, or non-linear topologies is future work (the design changes only one config line to swap models).
- Structural asymmetry of F1/F7: these failures are only possible under specialization, so they are reported as a newly introduced failure mode, not as a claim that specialization is worse overall.
- Heuristic failure classifiers: F1–F9 are automated heuristics; a blind manual re-label of a failure sample would strengthen the behavioral claims.
- Task shape: retrieval is deterministic and tasks are of moderate difficulty, which likely explains the high, saturated accuracy in both conditions; effects on messier, retrieval-limited tasks remain open.
- Residual nondeterminism: seeding reduces but does not fully eliminate runtime nondeterminism.
- The secondary GEE logistic model agrees with the primary paired Wilcoxon analysis on the reliability null; both are reported for transparency.

7. Conclusion

This report set out to isolate the causal effect of responsibility specialization in LLM-based multi-agent systems, a mechanism that is pervasive in open-source practice (Part 1: 333 systems mined; specialization common; LLM architecture judgments shown to be biased and needing evidence-grounding) yet rarely studied in isolation. Through a pre-registered, within-task controlled experiment (Part 2: 300 runs) in which specialization was the sole manipulated variable, we found that specialization preserves task accuracy (86.7% vs 86.0%, not significant) while significantly reducing token cost (~29%) and latency (~23%), at the cost of frequent role-boundary violations that were benign to correctness in this setting. The practical takeaway is that responsibility specialization should be understood chiefly as an efficiency and modularity mechanism rather than a reliability mechanism in short pipelines, and that role boundaries expressed only through instructions will be crossed often enough that safety-critical adherence must be enforced structurally. Future work should extend the same controlled methodology across models, agent counts, and topologies, and to tasks where out-of-role behavior can actually damage the outcome.

References

Note: the following are indicative references representative of the surveyed literature and this project's internal artifacts; specific citations should be finalized against the reading list.

- [1] Science of Scaling Multi-Agent Systems — on the effect of scaling agents and system organization (more agents do not automatically improve performance).
- [2] Why Do Multi-Agent LLM Systems Fail? — failure mechanisms in multi-agent systems (failures emerge from agent interactions and coordination).
- [3] MAESTRO — system-level multi-agent evaluation (architecture and coordination affect system behaviour).
- [4] AgentRx & AgentArc — agent architecture, behaviour, and design (agent organization as an important system-level factor).
- [5] Wu, Q., et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.
- [6] Hong, S., et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.
- [7] Li, G., et al. CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society.
- [8] Shinn, N., et al. Reflexion: Language Agents with Verbal Reinforcement Learning.
- [9] Internship Research Project 2026, Part 1: Empirical GitHub Characterization of Open-Source LLM Multi-Agent Systems (this project; 504 repositories, 333 systems, 1,087 agents).
- [10] Internship Research Project 2026, Part 2: Pre-registration and Results, 'Causal Effect of Responsibility Specialization in LLM Multi-Agent Systems' (this project).